Statistical methods for ML Report Clustering and Initialization

Matteo Bisotti
Department of Computer Science, University of Milan

Nicolò Cesa-Bianchi
Luigi Foscari
Emmanuel Esposito

8 May 2026

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.

1 Introduction

This report aims to present the work carried out for the final project of the Statistical Methods for Machine Learning course, taught by Professor Nicolò Cesa-Bianchi at the University of Milan.

The main goal of this project is to investigate the impact of initialization on clustering performance, with a particular focus on the `k-means` algorithm and its variant, `k-means++` [1]. Clustering is an unsupervised learning technique that consists of grouping similar data points into sets called clusters. The goal is to identify structures in the data, so that elements within the same cluster are more similar to each other than to elements assigned to different clusters.

The code developed for this project is available in a public repository [2].

1.1 Theoretical background

However, the `k-means` algorithm is well known to be highly sensitive to initialization, often converging to poor local minima. This instability makes the analysis of initialization strategies particularly relevant.

For the `k-means` problem, we are given a set of data points $\{x_1, \ldots, x_n\} \subset \mathbb{R}^d$ and a fixed number of clusters k. The goal is to find a set of centroids $\{c_1, \ldots, c_k\}$ that minimizes the potential function, commonly referred to as inertia:

$$\phi = \sum_{i=1}^{n} \min_{j \in \{1,\ldots,k\}} ||x_i - c_j||^2$$

This quantity measures the total squared distance between each data point and its closest centroid. Therefore,

- low value of inertia $\rightarrow$ well separated clusters;
- high value of inertia $\rightarrow$ spread out clusters.

Choosing these centers defines a clustering for each cluster, we set one cluster to be the set of data points that are closer to that center than to any other.

The `K-means` method works as follows:

1. arbitrarly choose k initial centers $\{c_1, \ldots, c_k\}$
2. for each $i \in \{1, \ldots, k\}$, set the cluster C_i to be the set of points in X that are closer to c_i than they are to c_j, for all $i \neq j$
3. for each $i \in \{1, \ldots, k\}$, set c_i to be the center of mass of all points in C_i: $c_i = \frac{1}{|C_i|} \sum_{x \in C_i} x$
4. repeat step 2 and 3 until the set of centers no longer changes.

With this algorithm, it's standard practice to choose the initial centers randomly from X. However, since the `k-means` objective is non-convex, the final solution strongly depends on this initial choice. It is guaranteed only to find a local optimum, which can often be quite poor.

To address this limitation, Arthur and Vassilvitskii proposed a variant of the original algorithm, called `k-means++`, which introduces a probabilistic initialization strategy. The first center is selected uniformly at random, while each subsequent center is chosen with probability proportional to its squared distance from the closest center already selected. This strategy encourages the initial centroids to be more spread out across the dataset, reducing the probability of poor initializations and improving the stability of the final clustering.

Let $D(x)$ denote the shortest distance from a data point x to the closest center, already chosen. `K-means++` works as follows:

1. choose an initial random center c_1 from X
2. choose the next center c_i selecting $c_i = x' \in X$ with probability $p(x') = \frac{D(x')^2}{\sum_{x \in X} D(x)^2}$
3. repeat step 2 until we chose a total of k centers
4. proceed as with the standard `K-means` algorithm.

In the paper [1], Arthur and Vassilvitskii prove that the expected value of the potential function obtained by `k-means++` satisfies the following bound:

$$\mathbb{E}[\phi] \leq 8(\ln k + 2)\phi_{\text{OPT}} = O(\log k)\phi_{\text{OPT}}$$

This result shows that the solution produced by `k-means++` is not arbitrarily far from the optimal clustering in expectation. Instead, its expected cost is bounded by a logarithmic factor in the number of clusters. Therefore, as k increases, the worst-case degradation with respect to the optimal solution grows only slowly.

This project aims to empirically investigate how a more careful initialization strategy affects the quality and stability of the solutions obtained. In this project, i implement both `k-means` and `k-means++` from scratch and evaluate their behaviour across multiple runs.

2 Dataset

The analysis is conducted on a synthetic datasets, where clustering structure is controlled, and real-world high-dimensional data (MNIST).

The synthetic dataset consists of 5 clusters ($k = 5$) in a two-dimensional space. Each cluster was generated by sampling points from a Gaussian distribution centered around a predefined location. The cluster centers were manually chosen in order to obtain a good clustering

$$[(0,0),\ (-8,5),\ (8,4),\ (-5,-7),\ (6,-6)]$$

The number of points assigned to the five clusters is respectively 120, 100, 110, 90, 100, for a total of 500 observations. This choice introduces a slight imbalance among the clusters, making the dataset less artificial.

Each group of points is distributed around its corresponding center, with the dispersion controlled by a parameter, the standard deviation. In this experiment, the standard deviation was set to $\sigma = 1.2$. The choice of working in two dimensions ($d = 2$) allows a direct visualization of both the generated data and the final clustering results. A fixed random seed was used to ensure the reproducibility of the experiments.

The distribution of the points in the synthetic dataset, with their corresponding clusters, is shown in Figure 1.

The real-world high-dimensional dataset used in this project is MNIST, it's a widely used benchmark in machine learning, consisting of 70000 grayscale images of handwritten digits (0–9), each represented as a 784-dimensional vector obtained by flattening 28×28 pixel images.

The dataset was loaded using `fetch_openml("mnist_784", version=1)` from the scikit-learn library. For computational reasons, instead of using the full dataset, a random subset of 2000 samples was selected. The subset selection was also performed using a fixed random seed, in order to guarantee reproducibility.

Before applying the clustering algorithms, the pixel values were converted to `float64` and normalized to the interval $[0, 1]$.

The MNIST dataset allows the algorithms to be evaluated in a more complex scenario than the synthetic dataset, since it is characterized by high dimensionality.

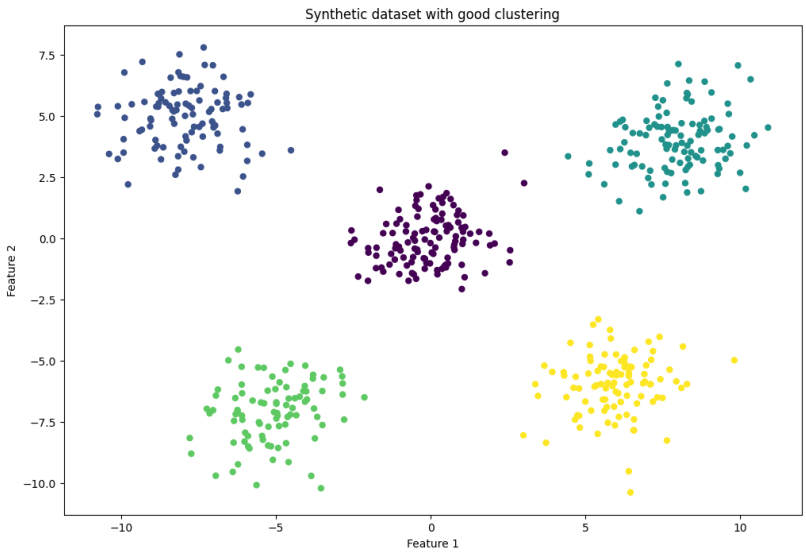

Figure 1: Distribution synthetic dataset

3 Methodology

This section describes the clustering algorithms used in the experiments. The main objective is to evaluate the impact of the initialization on the quality of the final solutions, by comparing the standard `k-means` algorithm with its `k-means++` variant. Both algorithms were implemented from scratch using Python and NumPy.

The `k-means` algorithm was implemented through a *KMeans* class, which includes the main components of the algorithm:

- centroid initialization (`initialize_centroids`);
- distance computation (`euclidean_distances`);

- assignment of points to clusters (`assign_clusters`);
- centroid update (`update_centroids`);
- inertia computation (`compute_inertia`);
- iterative optimization procedure (`fit`).

In particular, the `fit` method implements Lloyd's iterative procedure. At each iteration, points are assigned to the closest centroid and the centroids are then updated as the mean of the assigned points. The algorithm stops when the shift of the centroids between two consecutive iterations falls below a fixed threshold `tol`, or when the maximum number of iterations `max_iter` is reached. The actual number of iterations performed is stored in the attribute `n_iter`.

The `k-means++` variant was implemented through a *KMeansPlusPlus* class, which directly extends the *KMeans* class. In particular, the method for centroids' initialization is overridden in order to implement the probabilistic initialization strategy of `k-means++`.

To evaluate the effect of initialization, both algorithms were run repeatedly on the same datasets for a fixed number of independent runs (`n_runs`), varying only the random seed. The experimental procedure was organized through an external loop that, for each run, creates a new model instance and stores:

- the final inertia value;
- the number of iterations;
- the assigned labels;
- the final centroids.

The experimental parameters were kept consistent across the two algorithms and across the datasets, in order to ensure a comparison. The only difference concerns the number of clusters k: for the synthetic dataset, $k = 5$ was used, consistently with the data generation process, while for MNIST, $k = 10$ was used, reflecting the ten digit classes in the dataset. The full experimental settings are reported in Table 1.

Hyperparameters	**Synthetic dataset**	**MNIST**
`n_clusters`	5	10
`n_runs`	50	50
`max_iter`	100	100
`tol`	1e-4	1e-4

Table 1: Settings for runs

4 Results

In this section, the experimental results are presented and analyzed with the aim of comparing the performance of `k-means` and `k-means++`. The comparison focuses mainly on the final inertia values and on the variability across different runs, in order to evaluate the stability and reliability of the two initialization strategies.

4.1 Synthetic dataset

The first experiment was conducted on the synthetic dataset with well-separated clusters. As reported in Table 2, `k-means++` achieves a significantly lower mean inertia than standard `k-means`, with a smaller standard deviation.

Metric	**K-Means**	**K-Means++**
Mean inertia	3846.6721	1775.2924
Std inertia	3114.8178	1135.4003
Min inertia	1445.1999	1445.1999
Max inertia	12211.0824	6733.3857
Mean iterations	7.24	3.88

Table 2: Comparison of `k-means` and `k-means++` on the synthetic dataset

Figure 2 provides a qualitative comparison between the best and worst runs obtained by the two algorithms. In the best case, both methods are able to correctly identify the underlying cluster structure. However, in the worst case, standard `k-means` produces a poorer solution, with a much higher inertia value.

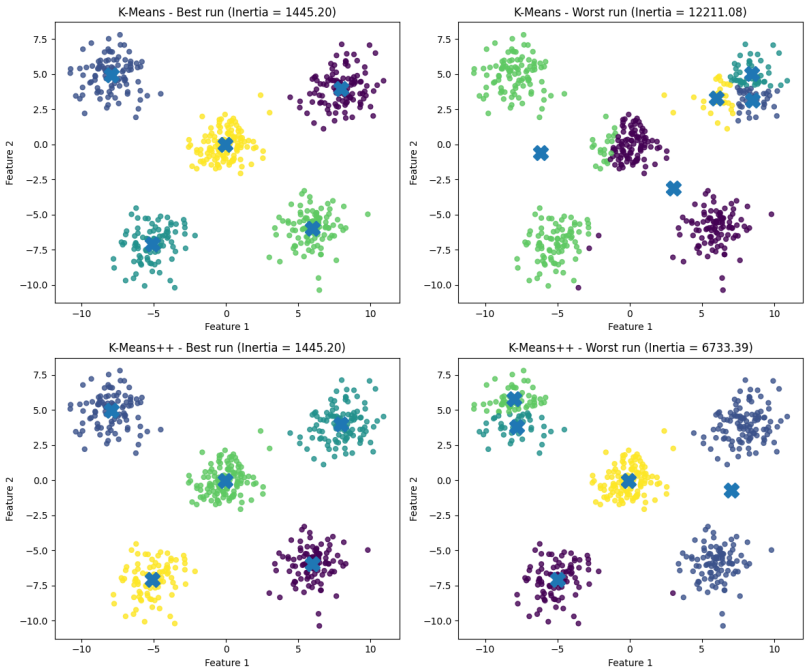

Figure 2: Best vs worst run on `k-means` and `k-means++`

The distribution of inertia values across the different runs, shown in Figure 3, shows a significant difference between two algorithms. Standard `k-means` exhibits high variability: while some runs converge to good solutions, and several other results with much larger inertia values. In contrast, `k-means++` produces a more concentrated distribution, with generally lower inertia values and fewer extreme outcomes.

The analysis of the number of iterations required for convergence, shown in Figure 4, also supports this result. On this dataset, `k-means++` tends to converge faster and with lower variability than standard `k-means`.

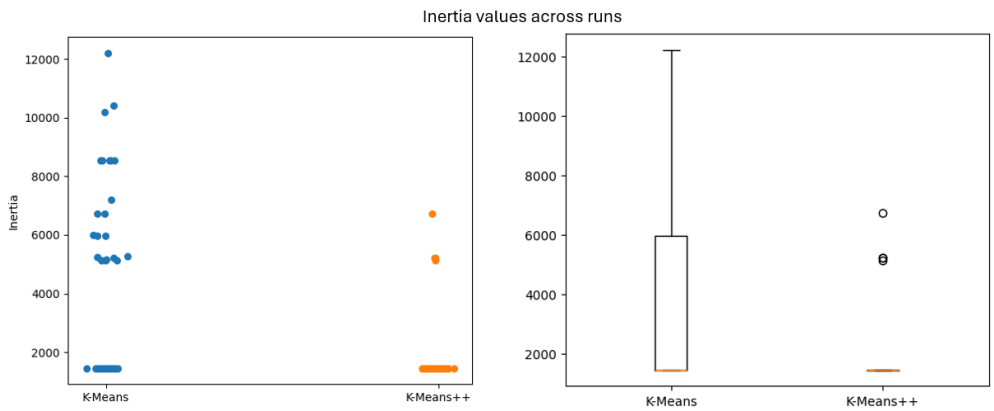

Figure 3: Inertia values on synthetic dataset

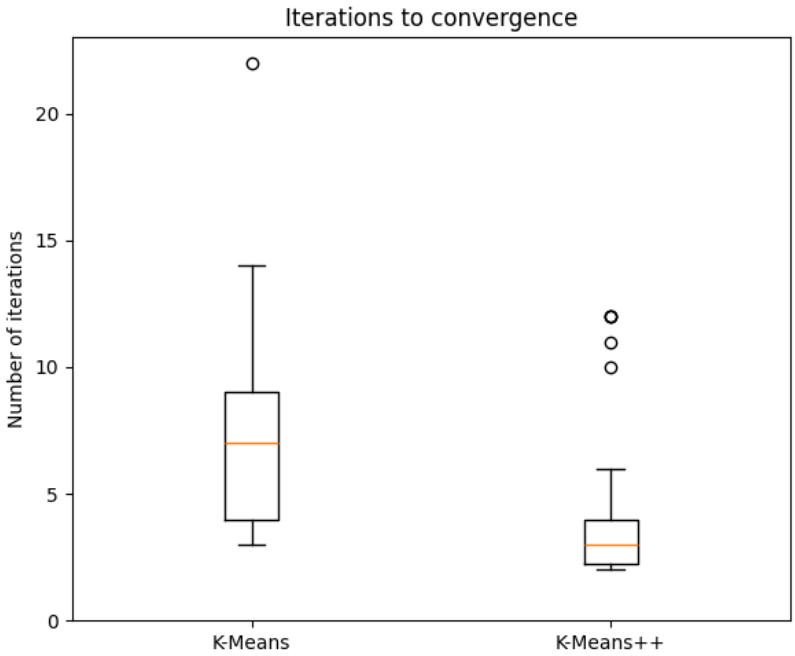

Figure 4: Number of iterations on synthetic dataset

4.2 MNIST dataset

The second experiment was conducted on the MNIST dataset, which represents a more challenging scenario due to its high dimensionality and less explicit cluster structure. Compared to the synthetic dataset, both algorithms produce much larger inertia values, as expected from the higher number of features and the complexity of the data representation.

As shown in Table 3 and Figure 5, the difference between `k-means` and `k-means++` is much less pronounced than in the synthetic case. The inertia values obtained by the two algorithms are concentrated in a relatively narrow range, indicating that the initialization strategy has a more limited effect on the final solution.

Regarding the number of iterations, both algorithms show higher variability compared to the synthetic experiment. Moreover, as shown in Figure 6, `k-means++` does not provide a systematic improvement in convergence speed.

Metric	**K-Means**	**K-Means++**
Mean inertia	78771.3477	78816.3558
Std inertia	257.8803	288.7149
Min inertia	78369.3144	78376.6201
Max inertia	79754.5337	79565.6179
Mean iterations	29.54	31.20

Table 3: Comparison of `k-means` and `k-means++` on the MNIST dataset

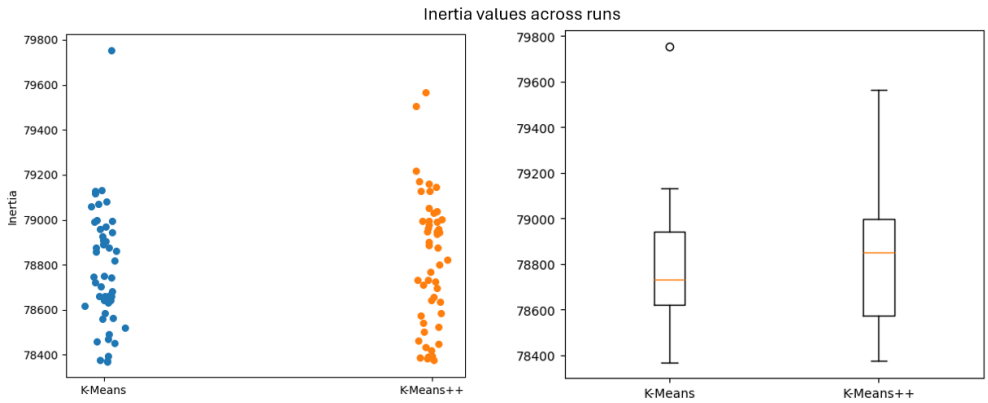

Figure 5: Inertia values on MNIST

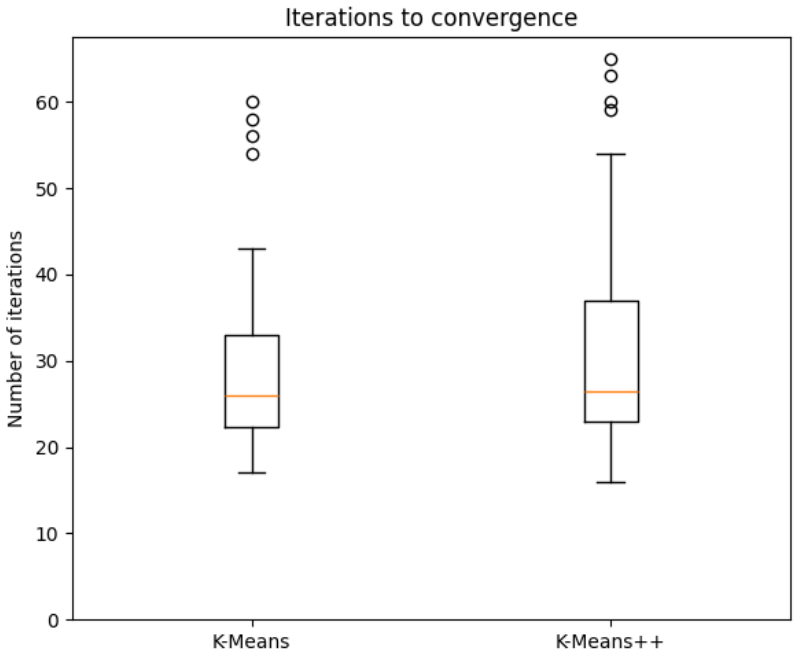

Figure 6: Number of iterations on MNIST dataset

4.3 Overlapping clusters

In this section, an additional experimental scenario is considered, in which the synthetic data exhibit overlapping clusters.

The dataset was generated using 5 clusters ($k = 5$) in a two-dimensional space, with the same cluster centers as in the previous experiment but with a larger standard deviation ($\sigma = 5$). This increases the dispersion of the points and produces a significant overlap between clusters. As shown in Figure 7, the resulting cluster structure is much less clearly defined.

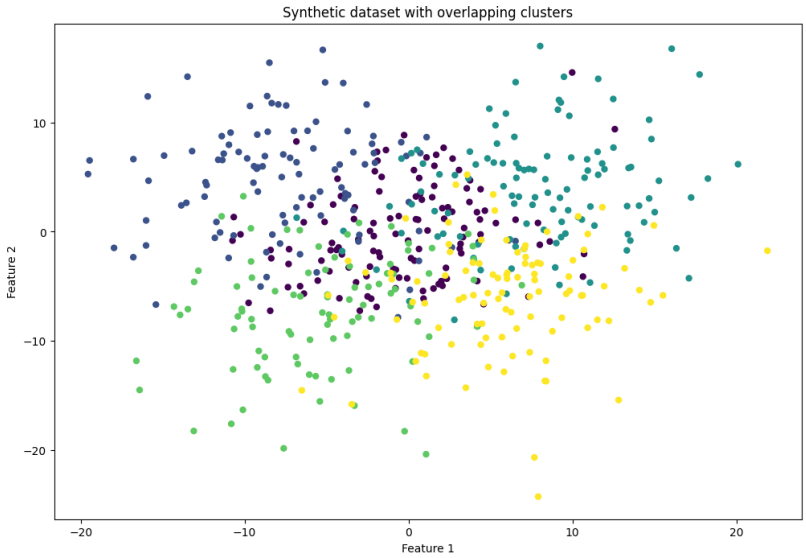

Figure 7: Distribution synthetic dataset with overlapping clusters

The algorithms were run using the same hyperparameters as in the previous experiments, reported in Table 4.

Hyperparameters	**Synthetic dataset with overlapping clusters**
`n_clusters`	5
`n_runs`	50
`max_iter`	100
`tol`	1e-4

Table 4: Settings for runs for dataset with overlapping clusters

The numerical results in terms of inertia and number of iterations for each algorithm are reported in Table 5.

Metric	**K-Means**	**K-Means++**
Mean inertia	15447.1556	15296.7253
Std inertia	470.2300	355.0358
Min inertia	15071.6371	15071.6371
Max inertia	16556.5887	16455.8327
Mean iterations	16.72	19.84

Table 5: Comparison of `k-means` and `k-means++` on dataset with overlapping clusters

Figure 8 shows a comparison between the best and worst run of each algorithm. In this setting, the difference between `k-means` and `k-means++` is much less pronounced than in the well-separated case.

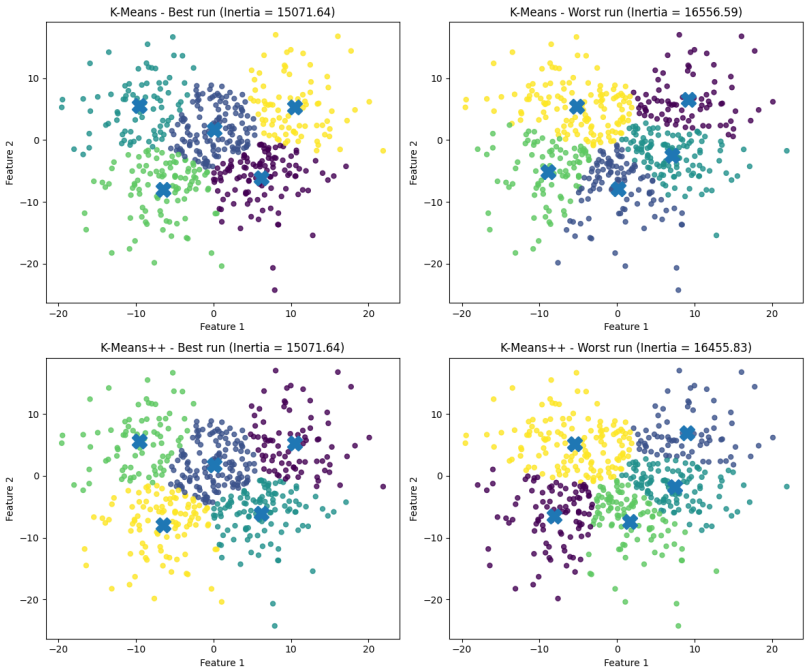

Figure 8: Best vs worst run on `k-means` and `k-means++` on dataset with overlapping clusters

As shown in Figure 9, the inertia values differ only moderately across runs. Both algorithms produce values concentrated in a relatively narrow range. Standard `k-means` exhibits slightly higher variability, whereas `k-means++` shows a more compact distribution around lower inertia values, although without an important improvement. Figure 10 reports the number of iterations required for convergence, showing that `k-means++` requires, on average, a slightly larger number of iterations than standard `k-means`.

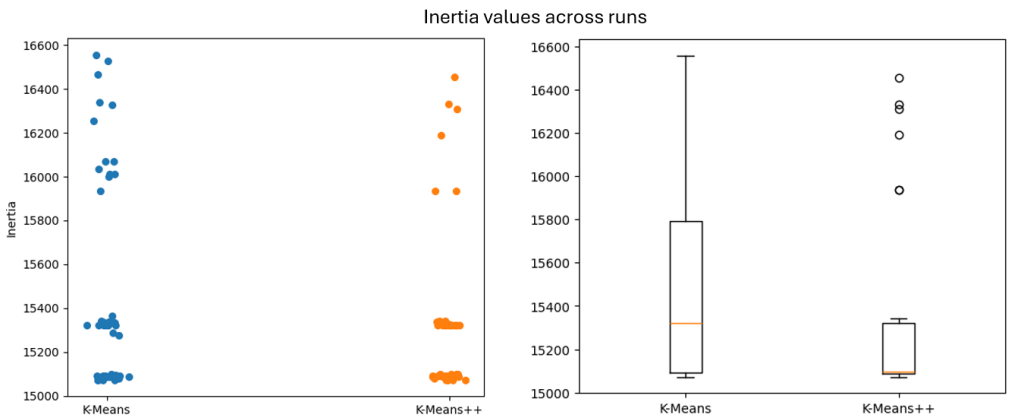

Figure 9: Inertia values on dataset with overlapping clusters

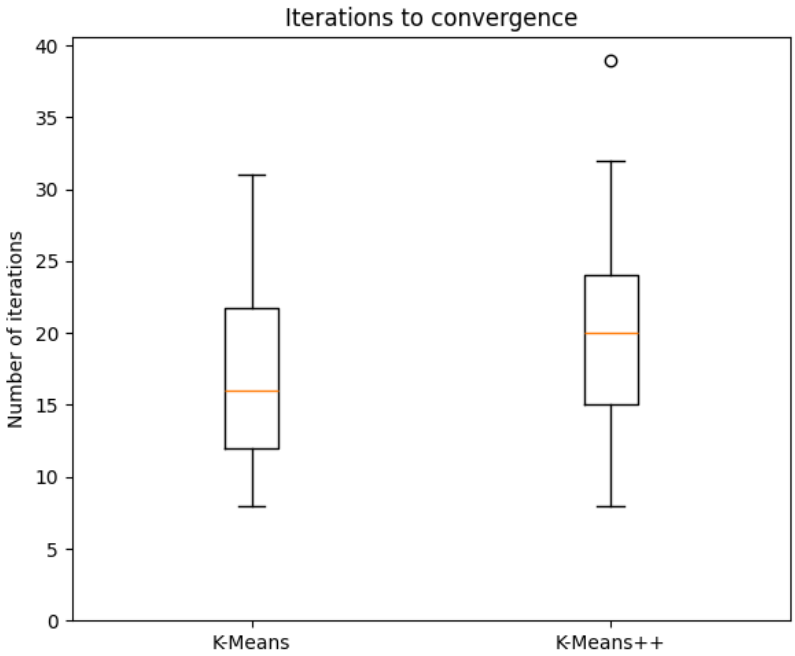

Figure 10: Number of iterations on synthetic dataset

5 Conclusions

In conclusion, the experiments show that initialization plays a central role in the behaviour of `k-means` and `k-means++`. However, its impact strongly depends on the structure of the dataset considered.

On the synthetic dataset with good clustering, `k-means++` achieves a clear improvement over standard `k-means`. The mean inertia is substantially lower, the variability across different runs is reduced, and convergence is reached in fewer iterations on average. This result is consistent with the idea that a more careful initialization allows the algorithm to start from a configuration that is already closer to a good final solution.

On the synthetic dataset with overlapping clusters, the advantage of `k-means++` becomes less pronounced. The inertia values obtained by the two algorithms are concentrated in a relatively narrow range, and the difference between the best and worst runs is smaller than in the previous experiment. This behaviour is consistent with the nature of the dataset: when clusters overlap significantly, there is no clear separation between groups, and different partitions may be comparable in terms of inertia. In this setting, the algorithm must still adjust the centroids in regions where cluster boundaries are ambiguous. As a result, even when starting from more spread-out initial centroids, `k-means++` may require more iterations to converge.

On the MNIST dataset, which is characterized by high dimensionality, the effect of initialization is further reduced compared to the complexity of

the data representation. In this case, `k-means++` does not provide a clear and systematic improvement over standard `k-means`, highlighting possible limitations of this approach in high-dimensional settings.

Overall, the results support the main idea of Arthur and Vassilvitskii's paper: a more careful centroid initialization strategy can significantly improve the quality and stability of `k-means`. At the same time, the experiments also show that initialization is not the only determining factor. Cluster separability, overlap and dimensionality all have a substantial influence on the final quality of the clustering.

References

[1] David Arthur and Sergei Vassilvitskii. k-means++: The advantages of careful seeding. In *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 1027–1035, 2007.

[2] Matteo Bisotti. Repository github. `https://github.com/MatteoBisotti/statML_project`, 2026.